

- **Exception handling:**
- In a language without exception handling –When an exception occurs, control goes to the operating system, where a message is displayed and the program is terminated.
- In a language with exception handling. –Programs are allowed to trap some exceptions, thereby providing the possibility of fixing the problem and continuing.

Basic Concepts

- An exception is any unusual event, either erroneous or not, detectable by either hardware or software, that may require special processing.
- The special processing that may be required after detection of an exceptions called exception handling.
- The exception handling code unit is called an **exception handler**

exception handling

- Exception handling is useful for dealing with exceptions that cannot be handled locally.
- Instead of showing an error status in the program, the exception handler transfers control to where the error can be handled.
- A function can throw exceptions or can choose to handle exceptions.
- **In the example, we are trying to divide a number by 0 . Here, this code generates an exception.**

Basic Concepts in Exception Handling (:

- **Definition** An *exception* is an unusual (but generally expected!) event that can be detected either by hardware or by software and that may require special processing.
- **Terminology** The special processing that may be required when an exception is detected is called *exception handling*. It is done by a code unit or segment called an *exception handler*. An exception is *raised* when its associated event occurs.

Approaches to handle exceptions in Java

try...catch block

finally block

throw and throws keyword

Java try...catch block

The try-catch block is used to handle exceptions in Java. Here's the syntax of try...catch block:

```
try {  
    // code  
}  
catch(Exception e) {  
    // code  
}
```

- **We have placed the code that might generate an exception inside the try block. Every try block is followed by a catch block.**
- **When an exception occurs, it is caught by the catch block. The catch block cannot be used without the try block.**

Exception handling using try...catch

```
class Main {  
    public static void main(String[] args) {  
  
        try {  
  
            // code that generate exception  
            int divideByZero = 5 / 0;  
            System.out.println("Rest of code in try block");  
        }  
  
        catch (ArithmeticException e) {  
            System.out.println("ArithmeticException => " + e.getMessage());  
        }  
    }  
}
```

Output

ArithmeticException => / by zero

- In the example, we are trying to divide a number by 0. Here, this code generates an exception.
- To handle the exception, we have put the code, `5 / 0` inside the try block. Now when an exception occurs, the rest of the code inside the try block is skipped.
- The catch block catches the exception and statements inside the catch block is executed.
- If none of the statements in the try block generates an exception, the catch block is skipped.

Exception Handling in Java:

The *finally* clause

- Sometimes, a process must be executed regardless of whether an exception is raised or not and handled or not.
- This occurs, for example, when a file must be closed or an external resource released, regardless of the outcome of the program.
- This is done by adding a *finally* clause at the end of the list of handlers, just after a complete try construct.
- The *finally* clause is executed in all cases whether or not *try* throws an exception, and whether or not it is caught by a *catch* clause.

Exception Handling in Java:

Binding Exceptions to Handlers

- An exception is **thrown** using the *throw* statement, for example:
`throw new MyException ("a message to specify the location of the error")`
- **Binding** If an exception is thrown in the compound statement of a try construct, it is bound to the first handler (catch function) immediately following the try clause whose parameter is the same class as the thrown object, or an ancestor of it. If a matching handler is found, the throw is bound to it and it is executed.

Python Exception Handling

- handle exceptions in Python using try, except, and finally statements

Different types of exceptions in python:

- In Python, there are several built-in Python exceptions that can be raised when an error occurs during the execution of a program.
- **SyntaxError:** This exception is raised when the interpreter encounters a syntax error in the code, such as a misspelled keyword, a missing colon, or an unbalanced parenthesis.
- **TypeError:** This exception is raised when an operation or function is applied to an object of the wrong type, such as adding a string to an integer.
- **NameError:** This exception is raised when a variable or function name is not found in the current scope.

- `IndexError`: This exception is raised when an index is out of range for a list, tuple, or other sequence types.
- `KeyError`: This exception is raised when a key is not found in a dictionary.
- `ValueError`: This exception is raised when a function or method is called with an invalid argument or input, such as trying to convert a string to an integer when the string does not represent a valid integer.

Try and Except Statement – Catching Exceptions

- Try and except statements are used to catch and handle exceptions in Python. Statements that can raise exceptions are wrapped inside the try block and the statements that handle the exception are written inside except block.

```
a = [1, 2, 3]
```

```
try:
```

```
    print ("Second element = %d" %(a[1]))
```

```
    print ("Fourth element = %d" %(a[3]))
```

```
except:
```

```
    print ("An error occurred")
```

Output

Second element = 2

An error occurred

- the statements that can cause the error are placed inside the try statement (second print statement in our case).
- The second print statement tries to access the fourth element of the list which is not there and this throws an exception. This exception is then caught by the except statement.

finally Keyword in Python

- Python provides a keyword finally, which is always executed after the try and except blocks. The final block always executes after the normal termination of the try block or after the try block terminates due to some exception. The code within the finally block is always executed.

try:

k = 5//0

print(k)

except ZeroDivisionError:

print("Can't divide by zero")

finally:

print('This is always executed')

Output:

Can't divide by zero

This is always executed

Advantages of Exception Handling:

- Improved program reliability: By handling exceptions properly, you can prevent your program from crashing or producing incorrect results due to unexpected errors or input.
- Simplified error handling: Exception handling allows you to separate error handling code from the main program logic, making it easier to read and maintain your code.
- Cleaner code: With exception handling, you can avoid using complex conditional statements to check for errors, leading to cleaner and more readable code.
- Easier debugging: When an exception is raised, the Python interpreter prints a traceback that shows the exact location where the exception occurred, making it easier to debug your code.

Disadvantages of Exception Handling:

- Performance overhead: Exception handling can be slower than using conditional statements to check for errors, as the interpreter has to perform additional work to catch and handle the exception.
- Increased code complexity: Exception handling can make your code more complex, especially if you have to handle multiple types of exceptions or implement complex error handling logic.
- Possible security risks: Improperly handled exceptions can potentially reveal sensitive information or create security vulnerabilities in your code, so it's important to handle exceptions carefully and avoid exposing too much information about your program.

Some design issues in exception handling include:

How exceptions occur and are handled

How an exception occurs and how the exception handler handles it is a design issue.

Handling exceptions at different levels

If the same exception occurs at different points in a program, it should be handled by the exceptional handler for that block, not the entire program.

Exception Types:

Choose appropriate exception types for different exceptional conditions. Using specific exception types allows for more precise error handling and enables developers to handle different exceptions differently. It is important to define custom exception types when necessary to capture specific exceptional conditions.

Exception Propagation:

Decide how exceptions should be propagated through the call stack. Should exceptions be caught and handled locally, or should they be propagated up the call stack to be handled at a higher level? This decision depends on the nature of the exception and the desired error handling strategy.

Error Reporting: Determine how exceptions should be reported to users or other components of the system. Clear and informative error messages can help users understand the cause of the error and take appropriate actions.

Design Issues for Exception Handling: Others

- Is finalization supported? That's the ability to complete some computations at the end of execution regardless of whether the program terminated normally or because of an exception.
- How are user-defined exceptions specified?
- Are there pre-defined exceptions?
- Should it be possible to disable predefined exceptions?
- If there are pre-defined exceptions, should there be default exception handlers for programs that do not provide their own handlers?
- Can pre-defined exceptions be explicitly raised?
- Are hardware-detectable errors treated as exceptions that may be handled?

Introduction to Event Handling (1)

- Event handling is similar to exception handling.
- Exceptions can be created either explicitly by user code or implicitly by hardware or a software interpreter. Events are created by *external actions*, such as user interactions through a Graphical User Interface.
- In event-driven programming, parts of the program are executed at completely unpredictable times, often triggered by user interactions with the executing program.

Introduction to Event Handling (2)

- An **event** is a notification that something specific has occurred, such as a mouse click on a graphical button.
- An **event handler** is a segment of code that is executed in response to the occurrence of an event.
- Event handling is useful in Web applications such as:
 - commercial applications where a user clicks on buttons to select merchandise,
 - Web form completion, where event handling is used to verify that no error or omission has been made in the completion of a form.
- Java supports two different approaches to presenting interactive displays to users: either from application programs or from applets.

Difference between exception handling and event handling

is how events are created and what they are used for:

- **Event handling**

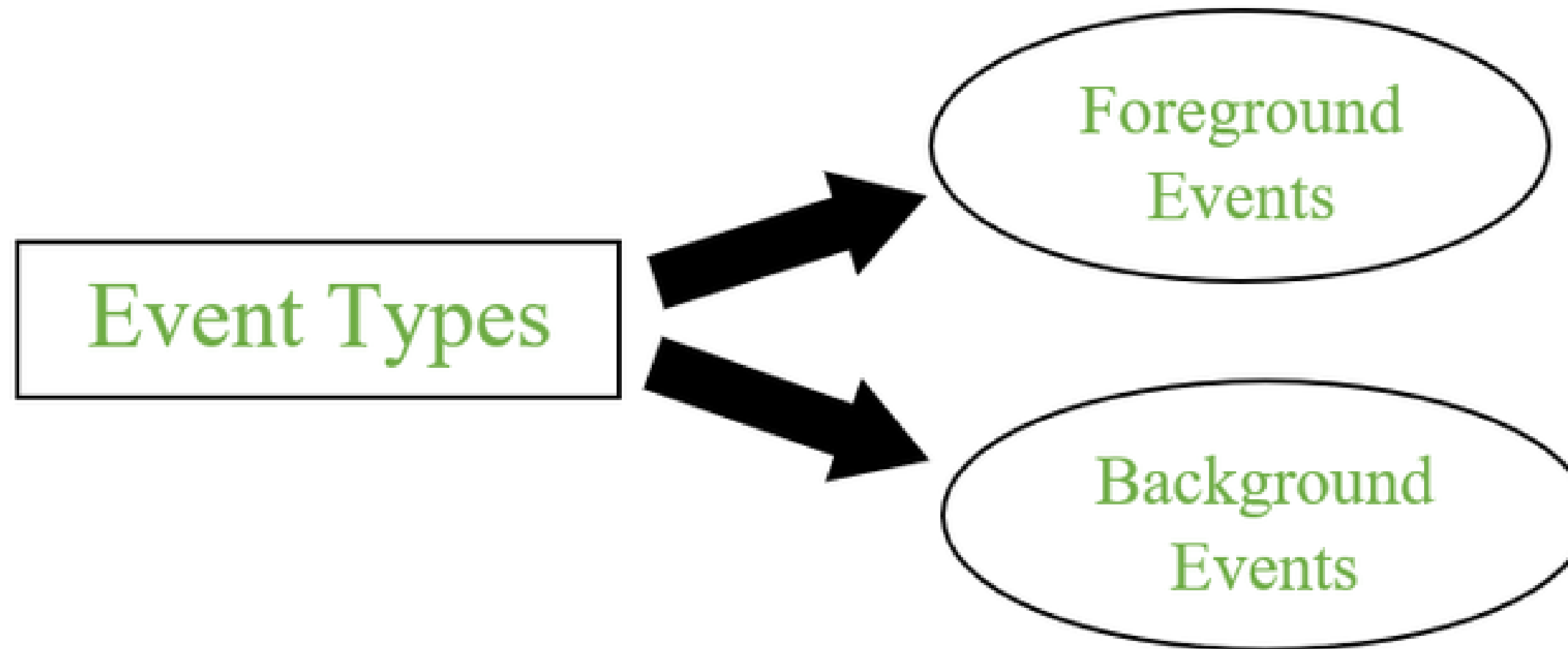
- Controls events and decides what to do when an event occurs. Events are created by external actions, like user interactions with a GUI. Event-driven programming executes parts of a program at unpredictable times, often triggered by user interactions.

- **Exception handling**

- Responds to unexpected events that occur when a computer program is running. Exceptions can be created by user code, hardware, or a software interpreter. Exception handling is used to avoid a program or system crashing. It's recommended to use exception handling when an event is exceptional and indicates an error, like an unexpected end-of-file.

- Here are some other common examples of an event handler:
- A notification pops up on a webpage when a new tab is opened.
- A form is submitted when the submit button is clicked.
- The background color changes on a mouse click.

Event Handling in Java



Foreground Events

- Foreground events are the events that require user interaction to generate, i.e., foreground events are generated due to interaction by the user on components in Graphic User Interface (GUI). Interactions are nothing but clicking on a button, scrolling the scroll bar, cursor movements, etc.

Background Events

- Events that don't require interactions of users to generate are known as background events. Examples of these events are operating system failures/interrupts, operation completion, etc.

Event Handling

- It is a mechanism to control the events and to decide what should happen after an event occur. To handle the events, Java follows the Delegation Event model.

Delegation Event model

- **It has Sources and Listeners.**
- **Source:** Events are generated from the source. There are various sources like buttons, checkboxes, list, menu-item, choice, scrollbar, text components, windows, etc., to generate events.
- **Listeners:** Listeners are used for handling the events generated from the source. Each of these listeners represents interfaces that are responsible for handling events.
- To perform Event Handling, we need to register the source with the listener.

Registering the Source With Listener

Syntax:

addTypeListener()

